

Convolutional Neural Networks

In the previous lecture, we built and trained a simple model to classify ASL images. The model was able to learn how to correctly classify the training dataset with very high accuracy, but, it did not perform nearly as well on validation dataset. This behavior of not generalizing well to non-training data is called [overfitting](#), and in this section, we will introduce a popular kind of model called a [convolutional neural network](#) that is especially good for reading images and classifying them.

Loading and Preparing the Data

The below cell contains the data preprocessing techniques we learned in the previous lectures. Execute it.

```
In [1]: import tensorflow.keras as keras
import pandas as pd
train_df=pd.read_csv('/content/drive/MyDrive/Colab Notebooks/sign_mnist_train.csv')
test_df=pd.read_csv('/content/drive/MyDrive/Colab Notebooks/sign_mnist_valid.csv')

# Separate out our target values
y_train = train_df['label']
y_test = test_df['label']
del train_df['label']
del test_df['label']

# Separate out our image vectors
x_train = train_df.values
x_test = test_df.values

# Turn our scalar targets into binary categories
num_classes = 24
y_train = keras.utils.to_categorical(y_train, num_classes)
y_test = keras.utils.to_categorical(y_test, num_classes)

# Normalize our image data
x_train = x_train / 255
x_test = x_test / 255
```

```
In [2]: from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
In [2]:
```

Reshaping Images for a CNN

The individual pictures in our dataset are in the format of long lists of 784 pixels:

```
In [3]: x_train.shape, x_test.shape
```

```
Out[3]: ((27455, 784), (7172, 784))
```

In this format, we don't have all the information about which pixels are near each other. Because of this, we can't apply convolutions that will detect features. Let's reshape our dataset so that they are in a 28x28 pixel format. This will allow our convolutions to associate groups of pixels and detect important features.

Note that for the first convolutional layer of our model, we need to have not only the height and width of the image, but also the number of [color channels](#). Our images are grayscale, so we'll just have 1 channel.

That means that we need to convert the current shape (27455, 784) to (27455, 28, 28, 1) . As a convenience, we can pass the [reshape](#) method a -1 for any dimension we wish to remain the same, therefore:

```
In [4]: x_train = x_train.reshape(-1,28,28,1)
        x_test = x_test.reshape(-1,28,28,1)
```

```
In [5]: x_train.shape
```

```
Out[5]: (27455, 28, 28, 1)
```

```
In [6]: x_test.shape
```

```
Out[6]: (7172, 28, 28, 1)
```

```
In [7]: x_train.shape, x_test.shape
```

```
Out[7]: ((27455, 28, 28, 1), (7172, 28, 28, 1))
```

Creating a Convolutional Model

These days, many data scientists start their projects by borrowing model properties from a similar project. Assuming the problem is not totally unique, there's a great chance that people have created models that will perform well which are posted in online repositories like [TensorFlow Hub](#) and the [NGC Catalog](#).

```
In [8]: from tensorflow.keras.models import Sequential
        from tensorflow.keras.layers import (
            Dense,
            Conv2D,
            MaxPool2D,
            Flatten,
            Dropout,
            BatchNormalization,
        )

        model = Sequential()
        model.add(Conv2D(75, (3, 3), strides=1, padding="same", activation="relu",
                          input_shape=(28, 28, 1)))
        model.add(BatchNormalization())
        model.add(MaxPool2D((2, 2), strides=2, padding="same"))
        model.add(Conv2D(50, (3, 3), strides=1, padding="same", activation="relu"))
        model.add(Dropout(0.2))
```

```
model.add(BatchNormalization())
model.add(MaxPool2D((2, 2), strides=2, padding="same"))
model.add(Conv2D(25, (3, 3), strides=1, padding="same", activation="relu"))
model.add(BatchNormalization())
model.add(MaxPool2D((2, 2), strides=2, padding="same"))
model.add(Flatten())
model.add(Dense(units=512, activation="relu"))
model.add(Dropout(0.3))
model.add(Dense(units=num_classes, activation="softmax"))
```

/usr/local/lib/python3.10/dist-packages/keras/src/layers/convolutional/base_conv.py:107: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Conv2D

These are our 2D convolutional layers. Small kernels will go over the input image and detect features that are important for classification. Earlier convolutions in the model will detect simple features such as lines. Later convolutions will detect more complex features. Let's look at our first Conv2D layer:

```
model.add(Conv2D(75, (3,3), strides = 1, padding = 'same'...))
```

75 refers to the number of filters that will be learned. (3,3) refers to the size of those filters. Strides refer to the step size that the filter will take as it passes over the image. Padding refers to whether the output image that's created from the filter will match the size of the input image.

BatchNormalization

Like normalizing our inputs, batch normalization scales the values in the hidden layers to improve training. [Read more about it in detail here.](#)

MaxPool2D

Max pooling takes an image and essentially shrinks it to a lower resolution. It does this to help the model be robust to translation (objects moving side to side), and also makes our model faster.

Dropout

Dropout is a technique for preventing overfitting. Dropout randomly selects a subset of neurons and turns them off, so that they do not participate in forward or backward propagation in that particular pass. This helps to make sure that the network is robust and redundant, and does not rely on any one area to come up with answers.

Flatten

Flatten takes the output of one layer which is multidimensional, and flattens it into a one-dimensional array. The output is called a feature vector and will be connected to the final classification layer.

Dense

We have seen dense layers before in our earlier models. Our first dense layer (512 units) takes the feature vector as input and learns which features will contribute to a particular classification. The second dense layer (24 units) is the final classification layer that outputs our prediction.

Summarizing the Model

This may feel like a lot of information, but don't worry. It's not critical that to understand everything right now in order to effectively train convolutional models. Most importantly we know that they can help with extracting useful information from images, and can be used in classification tasks.

Here, we summarize the model we just created. Notice how it has fewer trainable parameters than the model in the previous notebook:

```
In [9]: model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	P
conv2d (Conv2D)	(None, 28, 28, 75)	
batch_normalization (BatchNormalization)	(None, 28, 28, 75)	
max_pooling2d (MaxPooling2D)	(None, 14, 14, 75)	
conv2d_1 (Conv2D)	(None, 14, 14, 50)	
dropout (Dropout)	(None, 14, 14, 50)	
batch_normalization_1 (BatchNormalization)	(None, 14, 14, 50)	
max_pooling2d_1 (MaxPooling2D)	(None, 7, 7, 50)	
conv2d_2 (Conv2D)	(None, 7, 7, 25)	
batch_normalization_2 (BatchNormalization)	(None, 7, 7, 25)	
max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 25)	
flatten (Flatten)	(None, 400)	
dense (Dense)	(None, 512)	2
dropout_1 (Dropout)	(None, 512)	
dense_1 (Dense)	(None, 24)	



Total params: 264,049 (1.01 MB)

Trainable params: 263,749 (1.01 MB)

Non-trainable params: 300 (1.17 KB)

Compiling the Model

We'll compile the model just like before:

```
In [10]: model.compile(loss="categorical_crossentropy", metrics=["accuracy"])
```

Training the Model

Despite the very different model architecture, the training looks exactly the same. Run the cell below to train for 20 epochs and let's see if the accuracy improves:

```
In [13]: model.fit(x_train, y_train, epochs=10, verbose=1, validation_data=(x_test, y_test))
```

Epoch 1/10
858/858 ————— 3s 4ms/step - accuracy: 1.0000 - loss: 1.8709e-04 - val_acc
uracy: 0.9575 - val_loss: 0.1847
Epoch 2/10
858/858 ————— 3s 3ms/step - accuracy: 0.9999 - loss: 3.9430e-04 - val_acc
uracy: 0.9664 - val_loss: 0.1172
Epoch 3/10
858/858 ————— 3s 4ms/step - accuracy: 1.0000 - loss: 9.6278e-05 - val_acc
uracy: 0.9492 - val_loss: 0.2599
Epoch 4/10
858/858 ————— 5s 4ms/step - accuracy: 1.0000 - loss: 2.5790e-04 - val_acc
uracy: 0.9366 - val_loss: 0.3302
Epoch 5/10
858/858 ————— 5s 3ms/step - accuracy: 1.0000 - loss: 1.3509e-04 - val_acc
uracy: 0.9637 - val_loss: 0.2111
Epoch 6/10
858/858 ————— 4s 5ms/step - accuracy: 1.0000 - loss: 3.9915e-05 - val_acc
uracy: 0.9611 - val_loss: 0.1676
Epoch 7/10
858/858 ————— 4s 4ms/step - accuracy: 0.9999 - loss: 2.6958e-04 - val_acc
uracy: 0.9346 - val_loss: 0.3395
Epoch 8/10
858/858 ————— 3s 4ms/step - accuracy: 1.0000 - loss: 2.4627e-05 - val_acc
uracy: 0.9435 - val_loss: 0.3383
Epoch 9/10
858/858 ————— 6s 4ms/step - accuracy: 1.0000 - loss: 5.3917e-05 - val_acc
uracy: 0.9343 - val_loss: 0.4216
Epoch 10/10
858/858 ————— 4s 4ms/step - accuracy: 1.0000 - loss: 3.9486e-05 - val_acc
uracy: 0.9434 - val_loss: 0.2999

```
Out[13]: <keras.src.callbacks.history.History at 0x782470190550>
```

In the context of training Convolutional Neural Networks (CNNs) or any other neural network, `batch_size` refers to the number of training samples used in one forward and backward pass. The batch size is an important hyperparameter that influences the efficiency and performance of the training process.

Key Points about Batch Size:

Memory Usage: Larger batch sizes require more memory. If you use a batch size that is too large, you might run out of memory, especially if you're using a GPU with limited capacity.

Training Speed: Smaller batch sizes lead to faster iterations (batches are processed quicker), but might require more epochs to converge. Larger batch sizes lead to slower iterations but often require fewer epochs to converge.

Generalization: Smaller batch sizes tend to provide a noisier estimate of the gradient, which can help the model generalize better. Larger batch sizes provide a smoother estimate of the gradient, which can lead to faster convergence but might not generalize as well.

Example in TensorFlow/Keras:

Here's an example of how to specify the `batch_size` when training a CNN model using TensorFlow/Keras:

```
In [14]: import numpy as np
from sklearn.model_selection import train_test_split
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

# Example image data
X = np.random.rand(1000, 64, 64, 3)
y = np.random.randint(0, 10, 1000)

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=4)

# Define a simple CNN model
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(64, 64, 3)),
    MaxPooling2D((2, 2)),
    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(64, activation='relu'),
    Dense(10, activation='softmax')
])

# Compile the model
model.compile(loss='sparse_categorical_crossentropy', metrics=['accuracy'])

# Train the model with a specific batch size
batch_size = 32 # You can change this value
model.fit(X_train, y_train, epochs=10, batch_size=batch_size, validation_data=(X_test,
```

Epoch 1/10

```
/usr/local/lib/python3.10/dist-packages/keras/src/layers/convolutional/base_conv.py:107:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
```

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
25/25 ----- 5s 128ms/step - accuracy: 0.1097 - loss: 2.6899 - val_accuracy: 0.1200 - val_loss: 2.3056
```

Epoch 2/10

```
25/25 ----- 0s 6ms/step - accuracy: 0.0977 - loss: 2.3044 - val_accuracy: 0.1100 - val_loss: 2.3026
```

Epoch 3/10

```
25/25 ----- 0s 6ms/step - accuracy: 0.1355 - loss: 2.3030 - val_accuracy: 0.1100 - val_loss: 2.3035
```

Epoch 4/10

```
25/25 ————— 0s 6ms/step - accuracy: 0.1209 - loss: 2.3059 - val_accuracy:
0.1100 - val_loss: 2.3030
Epoch 5/10
25/25 ————— 0s 6ms/step - accuracy: 0.1247 - loss: 2.2903 - val_accuracy:
0.1100 - val_loss: 2.3188
Epoch 6/10
25/25 ————— 0s 6ms/step - accuracy: 0.1394 - loss: 2.2908 - val_accuracy:
0.1100 - val_loss: 2.3329
Epoch 7/10
25/25 ————— 0s 6ms/step - accuracy: 0.1916 - loss: 2.2270 - val_accuracy:
0.1100 - val_loss: 2.3094
Epoch 8/10
25/25 ————— 0s 6ms/step - accuracy: 0.3045 - loss: 2.1578 - val_accuracy:
0.0950 - val_loss: 2.3201
Epoch 9/10
25/25 ————— 0s 6ms/step - accuracy: 0.3511 - loss: 2.0326 - val_accuracy:
0.0850 - val_loss: 2.3415
Epoch 10/10
25/25 ————— 0s 6ms/step - accuracy: 0.4052 - loss: 1.8764 - val_accuracy:
0.1100 - val_loss: 2.3852
```

```
Out[14]: <keras.src.callbacks.history.History at 0x7823f0917af0>
```

In this example, the `batch_size` is set to 32. This means that during training, the model will process 32 samples at a time before updating the weights. You can experiment with different batch sizes to see what works best for your specific problem and hardware constraints.

Choosing the Right Batch Size:

Small Batch Sizes (e.g., 16, 32):

These can lead to better generalization and might be more suited for complex models or datasets with a lot of variance.

Large Batch Sizes (e.g., 128, 256):

These can speed up training if you have the computational resources and might be more suited for simpler models or larger datasets with less variance.

Ultimately, the choice of batch size often comes down to a trade-off between training speed, model performance, and available hardware resources. Experimenting with different batch sizes and monitoring the model's performance can help determine the optimal batch size for your specific application.

Discussion of Results

It looks like this model is significantly improved! The training accuracy is very high, and the validation accuracy has improved as well. This is a great result, as all we had to do was swap in a new model.